

littleOS: A Preemptive RTOS for the NXP FRDM-IMX91

Built upon a Custom Bare-Metal SDK

Shreekar Mane (B23CS1069)
Tavishi Srivastava (B23CS1101)
Yadav Karan Subhashchandra (B23EE1103)
Nallagonda Dharshan (B23EE1044)

April 18, 2026

Abstract

This report outlines the design and implementation of **littleOS**, a preemptive Real-Time Operating System (RTOS) built from scratch for the AArch64 Cortex-A CPU. We developed littleOS primarily to validate our custom bare-metal Software Development Kit (SDK) for the NXP FRDM-i.MX91 (SDK Report: <https://drive.google.com/file/d/12xxYnioYeXQkK-ZzKBfo6vJN43De5bmC/view?usp=sharing>). It acts as a practical proof-of-concept, tying together our memory-mapped peripheral drivers, hardware timer configurations, and exception-handling routines.

The core of littleOS features a custom context-switching engine, true thread blocking, and hardware-level atomic operations to prevent memory corruption. It supports dynamic scheduling (Round Robin, Priority, and Earliest Deadline First) and includes an interactive Command Line Interface (CLI). This report details the underlying OS mechanics, the thread mailbox system, and the modular task registry we built to run user applications.

The complete source code for both the bare-metal SDK and littleOS is available at: https://github.com/shreekar2005/BareMetal_FRDM_IMX91.

Contents

1	Introduction to littleOS	2
1.1	Building on the SDK	2
1.2	The System Boot Sequence	2
2	RTOS Core Architecture: How Multitasking Works	4
2.1	The Context Switching Engine (ARM Generic Timer)	4
2.2	Thread States and True Blocking (Sleep)	4
2.3	Scheduling Algorithms (RR, Priority, EDF)	5
2.4	Thread Mailboxes: Isolated Argument Passing	5
2.5	Hardware Atomics and Mutexes	6
3	The Command Line Interface (CLI)	7
3.1	Core System Commands	7
3.1.1	<code>help</code> / <code>?</code> : The System Directory	7
3.1.2	<code>kill</code> <code><id></code> and <code>kill all</code> : Thread Assassination	9
3.1.3	<code>sched</code> : Dynamic Scheduler Switching	9
3.1.4	<code>datetime</code> : RTC and Temporal Management	9
3.1.5	<code>clear</code> , <code>reboot</code> , and <code>shutdown</code>	9
4	Dynamic Tasks & Hardware Interfacing	11
4.1	The Auto-Loading Task Registry (<code>build_tasks.py</code>)	11
4.2	Task: <code>led</code> (Concurrency-Safe GPIO Control)	11
4.3	Task: <code>sonar</code> (Real-Time Proximity Driver)	11
4.4	Task: <code>esp</code> (Wi-Fi Configuration & Control)	12
4.5	Diagnostics: <code>statistics</code> , <code>echo</code> , and <code>race</code>	12

Chapter 1

Introduction to littleOS

1.1 Building on the SDK

Writing standalone drivers for components like timers, GPIO pins, and serial controllers is sufficient to prove we can talk to the i.MX91 hardware. However, to truly test the stability and interoperability of our bare-metal SDK, we needed a more demanding application. We built littleOS to stress-test our foundation.

littleOS relies heavily on the low-level drivers we implemented:

- **LPUART:** Drives the interactive CLI and manages asynchronous Wi-Fi data from the ESP8266.
- **SYS_CTR:** The ARM Generic Timer generates the precise hardware interrupts that drive our scheduler.
- **GIC:** The Generic Interrupt Controller manages priority routing between our scheduler ticks and UART data arrivals.
- **GPIO:** Used to test concurrency-safe physical outputs, like our multi-threaded LED blinker and sonar sensor tasks.

By pulling these drivers into a unified RTOS, littleOS upgrades the i.MX91 from a simple bare-metal loop into a multitasking edge node.

1.2 The System Boot Sequence

When U-Boot hands off execution to our bare-metal `_start` code, littleOS goes through a strict initialization phase to stabilize the hardware and allocate memory before accepting any user input.

The boot process logs directly to the serial terminal:

```
[littleOS] Welcome TO littleOS RTOS :)
[Boot] Initializing Scheduler...
[Boot] Initializing Task Registry...
[Boot] Registering ESP8266 IRQ...
[Boot] Creating System RTC Daemon...
[Boot] Creating WiFi Listener Thread...
[Boot] Creating CLI Thread...
[Boot] Setup complete! Starting Threads...
```

During this sequence, the OS statically allocates 16KB stacks for the background daemons and registers all available tasks from the `build_tasks.py` output into the internal registry. Once the GIC arms the hardware IRQs for the ESP8266 and the scheduler timer, the kernel sets the initial `cpustate_ptr` for the CLI thread, triggers the first context switch, and hands control over to the user.

Chapter 2

RTOS Core Architecture: How Multitasking Works

2.1 The Context Switching Engine (ARM Generic Timer)

Since we are running on a single-core AArch64 processor, true parallel execution isn't possible. Instead, littleOS creates the illusion of multitasking by rapidly hot-swapping threads.

We configured the ARM Generic Timer to trigger a hardware interrupt at regular intervals (e.g., every 1ms). When the timer fires, execution traps to our IRQ handler. Our context switcher then forcibly saves the active thread's state by pushing all 31 general-purpose registers (x0 through x30), the Exception Link Register (ELR_EL2), and the Saved Program Status Register (SPSR_EL2) onto that thread's stack. This consumes exactly 800 bytes to preserve a perfect snapshot of the thread's execution.

The scheduler then picks the next thread in the queue, pops its 800-byte context off its respective stack, and tells the CPU to resume execution right where it left off.

2.2 Thread States and True Blocking (Sleep)

A major issue in basic embedded programming is relying on "busy-wait" loops (empty `while` loops) for delays, which waste valuable CPU cycles.

To solve this, littleOS uses a true blocking architecture. We track every thread using a state machine (`STATE_NEW`, `STATE_READY`, `STATE_WAIT_BLOCK`, `STATE_TERMINATE`). When a thread calls `thread_sleep(delay_ms)`, the OS calculates the exact future hardware tick when it should wake up, flags it as `STATE_WAIT_BLOCK`, and immediately yields the CPU. The scheduler ignores sleeping threads entirely, saving 100% of the CPU utilization for tasks that actually need to run.

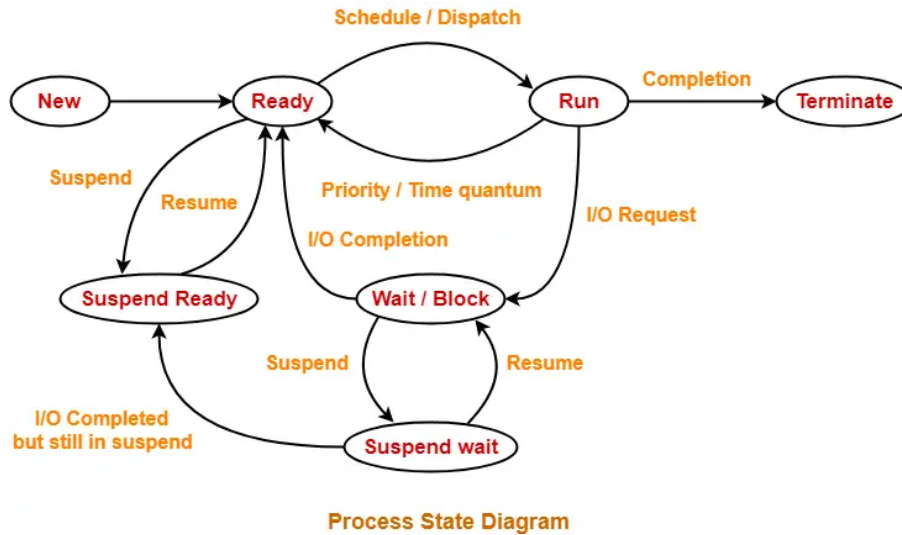


Figure 2.1: Task/Thread state machine transitions

2.3 Scheduling Algorithms (RR, Priority, EDF)

We designed the scheduling engine to be dynamic, allowing developers to swap between three different algorithms on the fly via the CLI depending on the system’s current needs:

- **Round Robin (RR):** The default mode. It treats all threads equally, looping through the ready queue and giving each a strict time-slice before preemption.
- **Fixed Priority (PRI):** Threads get a static priority from 0 to 255 (0 being highest). The scheduler always executes the highest-priority thread available. If there’s a tie, it round-robins between them.
- **Earliest Deadline First (EDF):** A dynamic algorithm for strict timing. Threads are assigned absolute deadline ticks. The scheduler continuously re-evaluates the queue to run the thread that is closest to missing its deadline.

2.4 Thread Mailboxes: Isolated Argument Passing

Early in development, we ran into a nasty race condition with the CLI. If a user spawned a thread with an argument (like `led blink red 100`), the thread read that string directly from the CLI’s input buffer. If the user typed a new command before the background thread finished parsing, the string was overwritten, crashing the task.

To fix this, we built a **Thread Mailbox System**. We allocated a 2D array: `char thread_arg_buffer[MAX_THREADS][CMD_BUFFER_SIZE]`. Now, when the CLI spawns a task, it copies the argument string into a dedicated 128-byte mailbox strictly assigned to that thread ID. This isolates the memory, allowing the CLI to instantly clear its buffer and take new input without corrupting background tasks.

2.5 Hardware Atomics and Mutexes

When multiple threads try to access the same hardware (like two threads printing to UART or sending Wi-Fi data), we need Mutual Exclusion (Mutexes).

However, a standard software boolean flag is dangerous in a preemptive OS—the scheduler might interrupt a thread halfway through checking the flag. For true thread safety, we used ARMv8 hardware atomics via the compiler’s `__atomic_compare_exchange_n` built-in, which maps to the AArch64 `LDXR` (Load Exclusive) and `STXR` (Store Exclusive) assembly instructions.

When a thread tries to lock a mutex, the CPU hardware monitors that exact memory address. If a context switch happens and another thread alters the lock, the `STXR` instruction explicitly fails, and our original thread knows to safely retry. This ensures our UART and Wi-Fi buffers are never scrambled.

Chapter 3

The Command Line Interface (CLI)

3.1 Core System Commands

When littleOS finishes booting, it spins up the essential background daemons (scheduler, ESP8266 IRQ, Wi-Fi listener) and drops the user into the CLI thread, as shown in Figure 3.1. From here, the user can interact directly with the board and run applications.

```
[littleOS] Welcome TO littleOS RTOS :)
[Boot] Initializing Scheduler...
[Boot] Initializing Task Registry...
[Boot] Registering ESP8266 IRQ...
[Boot] Creating System RTC Daemon...
[Boot] Creating WiFi Listener Thread...
[Boot] Creating CLI Thread...
[Boot] Setup complete! Starting Threads...
```

Figure 3.1: CLI prompt immediately after littleOS initialization

3.1.1 help / ?: The System Directory

The `help` command is the main navigation tool for the OS. The output is structured into three segments:

```
[CLI-Thread] > help
[CLI-Thread] Available Commands:
[CLI-Thread] kill <thread_id>all - Terminate a specific thread or all threads
[CLI-Thread] clear - Clear the terminal screen
[CLI-Thread] reboot/restart/reset - Hardware reboot
[CLI-Thread] shutdown/poweroff - Hardware poweroff (NOT SHUTTING DOWN HARDWARE!!!)
[CLI-Thread] sched [rr|p|edf] - Change RTOS scheduler algorithm
[CLI-Thread] datetime - System RTC time management
[CLI-Thread]
[CLI-Thread] Dynamic Tasks (Auto-Loaded from tasks/):
[CLI-Thread] print100 - Print 0's
[CLI-Thread] race - Race Condition
[CLI-Thread] echo - DBG ECHO
[CLI-Thread] statistics - Show/Send Stats
[CLI-Thread] print100X - Print X's
[CLI-Thread] esp - ESP Commands
[CLI-Thread] aprint100A - AtomicPrint A's
[CLI-Thread] led - LED Blink
[CLI-Thread] sonar - Sonar Proximity
[CLI-Thread] Syntax: <taskname> [-s] [-n <executions>] [-per <period_ms>] [-pri <priority>] [-d <deadline_ms>]
[CLI-Thread] -s : Silent mode (default: false)
[CLI-Thread] -n : Number of executions (default: 1, -1 for infinite)
[CLI-Thread] -per : Periodicity in ms (default: 0 for one-shot)
[CLI-Thread] -pri : Priority (default: 128, lower is higher priority)
[CLI-Thread] -d : Relative deadline in ms (default: -1 for no deadline)
```

Figure 3.2: Output of the "help" command

1. Available Commands

These handle core system configurations. You can reboot the board, clear the terminal, force-kill threads, or swap the scheduler logic.

A notable utility here is **datetime** (Figure 3.3). Since the bare-metal board doesn't have an NTP client, this command lets us view the RTC, manually set it, or sync it over TCP by querying an external server.

```
[CLI-Thread] > datetime
[DATEIME-Thread] Invalid argument. Usage:
[DATEIME-Thread]   datetime show                               (Prints current time)
[DATEIME-Thread]   datetime sync <server_ip> <port>          (Fetches real time via TCP)
[DATEIME-Thread]   datetime set <hh:mm:ss> <dd:mm:yyyy>      (Manually update RTC)
```

Figure 3.3: Utilities available under the **datetime** command

2. Dynamic Tasks

This lists all the user applications currently compiled into the OS.

For example, the **print100o** and **print100X** tasks simply print their respective characters 100 times. As seen in Figure 3.4, when we run both simultaneously under Round Robin scheduling, the OS perfectly alternates between printing 'X' and 'o' as it swaps their time-slices.

[illegible]

Figure 3.4: **print100X** and **print100o** demonstrating concurrent execution

Conversely, the **aprint100A** task acts as an atomic test. It temporarily disables the OS scheduler, prints 'A' 100 times in one shot, and then re-enables scheduling. We can actually verify this behavior using the **statistics show** command (Figure 3.6). While the standard print tasks take over 6000ms to complete because they yield the CPU, **aprint100A** finishes in exactly 5000ms because it never goes idle.

```
[CLI-Thread] > statistics
[STATISTICS-Thread] Usage:
[STATISTICS-Thread]   statistics show                (prints statistics to debug console)
[STATISTICS-Thread]   statistics sendto <target_ip> <target_port> (Sends to custom IP and Port)
```

Figure 3.5: Output of the **statistics** command

```
[CLI-Thread] > statistics show

[CLI-Thread] >
[STATISTICS-Thread] UPTIME:
[STATISTICS-Thread] 0 hrs : 4 min : 59 sec
[STATISTICS-Thread]
[STATISTICS-Thread] SCHEDULER:
[STATISTICS-Thread] Algorithm: Round Robin
[STATISTICS-Thread]
[STATISTICS-Thread] THREADS:
[STATISTICS-Thread]


| ID | Name            | State | Pri | Dead | Per | Targ | Done | TAT(ms) |
|----|-----------------|-------|-----|------|-----|------|------|---------|
| 1  | Print o's       | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 2  | Race Condition  | TERM  | 128 | -1   | 0   | 1    | 1    | 3961    |
| 3  | DBG ECHO        | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 4  | Show/Send Stats | RUN   | 128 | -1   | 0   | 1    | 1    | 642     |
| 5  | Print K's       | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 6  | ESP Commands    | TERM  | 128 | -1   | 0   | 1    | 1    | 48      |
| 7  | AtomicPrint A's | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 8  | LED Blink       | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 9  | Sonar Proximity | NEW   | 128 | -1   | 0   | 0    | 0    | 0       |
| 10 | RTC_Daemon      | BLOCK | 128 | -1   | 0   | -1   | 1    | 0       |
| 11 | ESpWifiListener | BLOCK | 128 | -1   | 0   | 1    | 1    | 0       |
| 12 | CLI             | BLOCK | 128 | -1   | 0   | 1    | 1    | 0       |
| 13 | Race-Slave1     | TERM  | 128 | -1   | 0   | 0    | 1    | 273     |
| 14 | Race-Slave2     | TERM  | 128 | -1   | 0   | 0    | 1    | 286     |
| 15 | Race-Slave3     | TERM  | 128 | -1   | 0   | 0    | 1    | 1655    |
| 16 | Race-Slave4     | TERM  | 128 | -1   | 0   | 0    | 1    | 1328    |


[STATISTICS-Thread]
[STATISTICS-Thread] ESP STATUS:
[STATISTICS-Thread] Reachable: True
[STATISTICS-Thread] Op Mode: 1
[STATISTICS-Thread] Router SSID: shree_tenda
[STATISTICS-Thread] Router MAC: cc2d21ebc8cb:c8
[STATISTICS-Thread] ESP IP: 192.168.0.101
[STATISTICS-Thread] ESP MAC: A8:3f:da:a3:b2:34
```

Figure 3.6: Detailed OS telemetry via **statistics show**

3. Task Execution Syntax & Flags

We wanted the CLI to feel like a real OS, so we added flags to let users control exactly how a task runs:

- **-s**: Runs the task in silent mode (suppresses terminal output).
- **-n <executions>**: Sets how many times a task loops (-1 for infinite).
- **-per <period.ms>**: Turns the task into a recurring periodic background job.
- **-pri <priority>**: Overrides the default thread priority.
- **-d <deadline.ms>**: Sets an execution deadline (used to test the EDF scheduler).

3.1.2 `kill <id>` and `kill all`: Thread Assassination

To manage rogue or infinite-looping tasks, we built a `kill` command. Passing `kill all` iterates through the registry and forces every active background thread into `STATE_TERMINATE`.

If the user passes a specific ID (e.g., `kill 5`), the CLI parses the number and maps it to our internal thread array. We included strict bounds checking here to prevent users from accidentally assassinating Thread 0 (the CLI thread), which would permanently brick the terminal.

3.1.3 `sched`: Dynamic Scheduler Switching

Typing `sched rr`, `sched pri`, or `sched edf` updates a global kernel flag. The next time the ARM timer fires, the context-switching engine reads this flag and instantly alters how it picks the next thread from the `STATE_READY` queue. This makes it incredibly easy to benchmark different real-time constraints without recompiling the OS.

3.1.4 `datetime`: RTC and Temporal Management

Because our bare-metal environment lacks DNS, `datetime sync <ip> <port>` uses a "Reverse Callback" trick. The board locks the Wi-Fi mutex, connects to our Python server over TCP, requests the time, and closes the connection. The Python server immediately opens a new connection back to the board and injects a formatted `datetime set` command right into the CLI, syncing the hardware clock.

3.1.5 `clear`, `reboot`, and `shutdown`

These commands talk directly to the hardware:

- **clear**: Sends standard ANSI escape codes (`\033[2J\033[H`) to flush the host PC's terminal buffer.
- **reboot**: Writes a specific magic sequence (`0xD928C520`) to the i.MX91's Hardware Watchdog (WDOG) registers to trigger a physical board reset.

- **shutdown:** Simulates a power-down by setting the SNVS LPCR register bits, disabling interrupts, and putting the ARM core into a permanent `wfi` (Wait For Interrupt) sleep.

Chapter 4

Dynamic Tasks & Hardware Interfacing

4.1 The Auto-Loading Task Registry (`build_tasks.py`)

We got tired of manually updating C arrays every time we wrote a new task, so we automated it. Now, developers just drop a `.c` file into the `tasks/` folder with a metadata comment on line 1 (`// Task_Name : <Name>`). During ‘make’, our `build_tasks.py` script scans the folder, parses the entry points, and auto-generates `autotasks.h` and `autotasks.c`. This links new apps into the CLI instantly.

4.2 Task: `led` (Concurrency-Safe GPIO Control)

The `led` task is a great example of hardware concurrency. Initially, if we spawned one thread to blink the LED at 1Hz, and another to blink it at 100Hz, both threads would fight over the GPIO pins in a massive race condition (what we called “Thread Warfare”).

To fix this, we used an “Update and Exit” architecture. Running a new `led blink` command simply updates a global `ledblink_delay_ms` variable and then kills itself. The original, already-running thread just reads the new delay value on its next cycle and adapts its speed seamlessly.

4.3 Task: `sonar` (Real-Time Proximity Driver)

For the `sonar` task, we hooked up an HC-SR04 ultrasonic sensor. To get clean acoustic timings, the driver actually pauses the OS scheduler for a few microseconds while pulsing the trigger pin, ensuring a context switch doesn’t corrupt the math.

We applied a median filter over 5 samples to handle hardware noise. The thread then manipulates the global LED delay variable based on distance: at our threshold (e.g., 100mm), the LED blinks every 1000ms. As an object gets closer to 0mm, the delay scales linearly down to 0ms, acting as a real-time proximity alarm.

```

[CLI-Thread] > sonar
[CLI-Thread] > [SONAR-Thread] Distance: 16.1 cm
[SONAR-Thread] Distance: 16.2 cm
[SONAR-Thread] Distance: 15.0 cm
[SONAR-Thread] Distance: 16.4 cm
[SONAR-Thread] Distance: 14.8 cm
[SONAR-Thread] Distance: 15.2 cm
[SONAR-Thread] Distance: 19.7 cm
[SONAR-Thread] Distance: 16.6 cm
[SONAR-Thread] Distance: 18.2 cm
[SONAR-Thread] Distance: 14.7 cm
[SONAR-Thread] Distance: 15.9 cm
[SONAR-Thread] Distance: 14.5 cm
[SONAR-Thread] Distance: 15.4 cm
[SONAR-Thread] Distance: 15.2 cm
[SONAR-Thread] Distance: 15.6 cm
[SONAR-Thread] Distance: 17.6 cm
[SONAR-Thread] Distance: 25.9 cm
[SONAR-Thread] Distance: 22.9 cm
[SONAR-Thread] Distance: 101.0 cm
[SONAR-Thread] Distance: 101.7 cm
[SONAR-Thread] Distance: 25.4 cm
[SONAR-Thread] Distance: 25.7 cm
[SONAR-Thread] Distance: 23.0 cm
[SONAR-Thread] Distance: 22.3 cm

```

Figure 4.1: Real-time, noise-filtered proximity calculations output by the Sonar thread

4.4 Task: esp (Wi-Fi Configuration & Control)

The **esp** task is our networking bridge. It allows the user to spin up the ESP8266 as an Access Point, connect to a router, or start a TCP server directly from the terminal.

```

[CLI-Thread] > esp
[ESP-Thread] Invalid argument.
[ESP-Thread] Usage:
[ESP-Thread]   esp sta-mode <ssid_name> <ssid_password> (Both required)
[ESP-Thread]   esp ap-mode [ssid_name] [ssid_password] (Default: littleOS / littleOS)
[ESP-Thread]   esp tcp-server [port_number] (Default: 8080)
[ESP-Thread]   esp echo <message> (Sends text to TCP clients)
[ESP-Thread]   esp reboot (Reboots the ESP8266 module)
[ESP-Thread]   esp status (Shows current mode, IP, MAC)

```

Figure 4.2: The **esp** task argument parser and usage syntax

One neat feature is the **echo** command. Instead of letting the CLI tokenizer break a sentence apart by spaces, we implemented a custom pointer offset that grabs the entire raw string following the command and blasts it over the TCP socket intact.

4.5 Diagnostics: statistics, echo, and race

We built a few diagnostic tasks to profile the OS:

- **race:** Intentionally spawns threads that fight over shared memory to prove why our LDXR/STXR hardware mutexes are necessary.
- **echo:** A loopback utility for debugging UART payloads.
- **statistics:** Computes thread turn-around times, states, and Wi-Fi data.

By running **statistics sendto <ip>**, the board formats this telemetry into a JSON payload and streams it to our external Python WebApp, turning littleOS into a fully monitored IoT node.

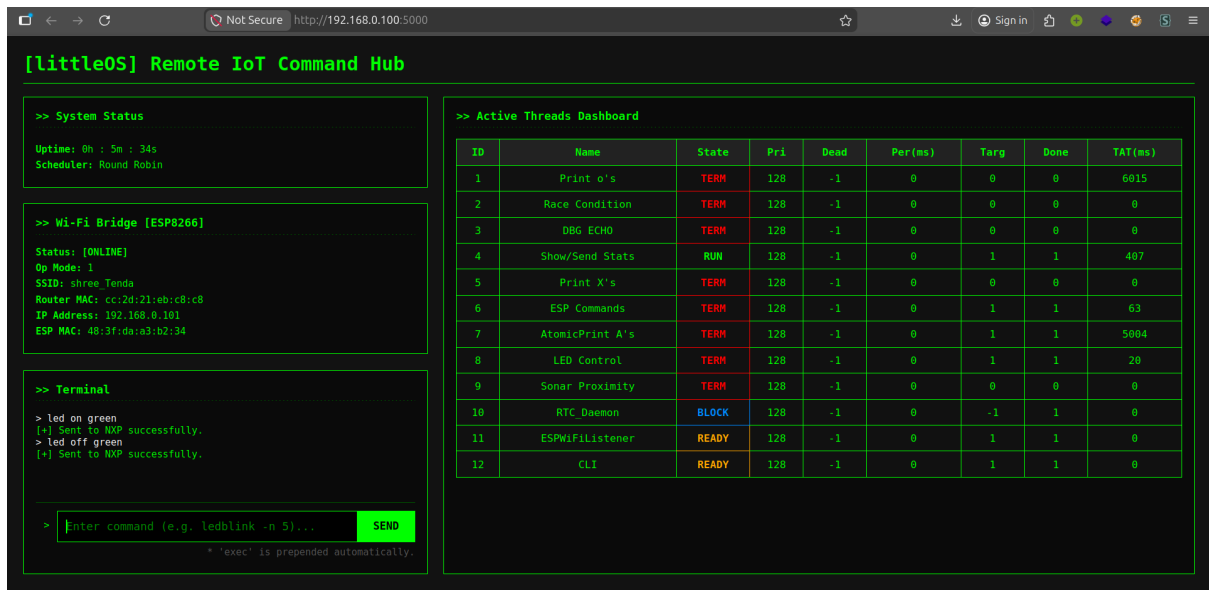


Figure 4.3: The littleOS Python Command Hub parsing JSON telemetry from the bare-metal board